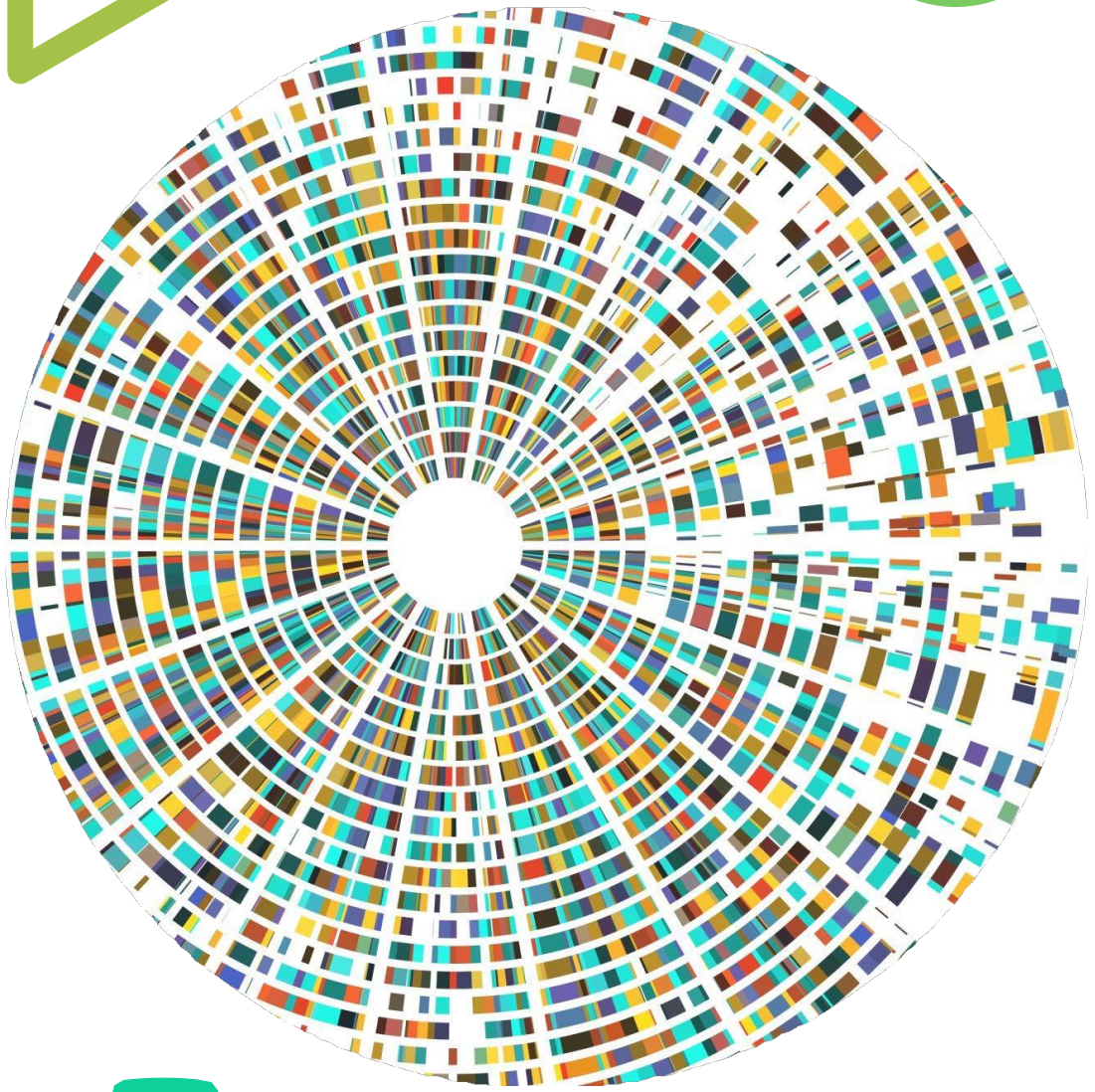
IES ALBARREGAS

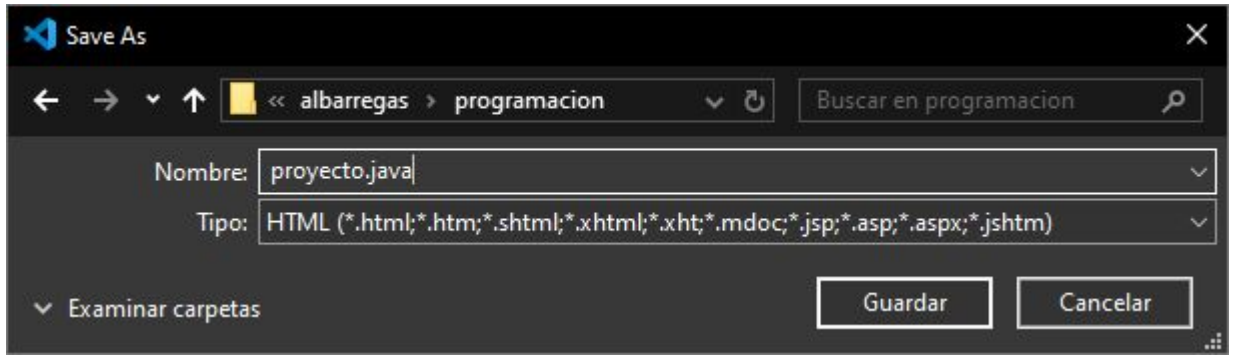
Gestión de repositorios

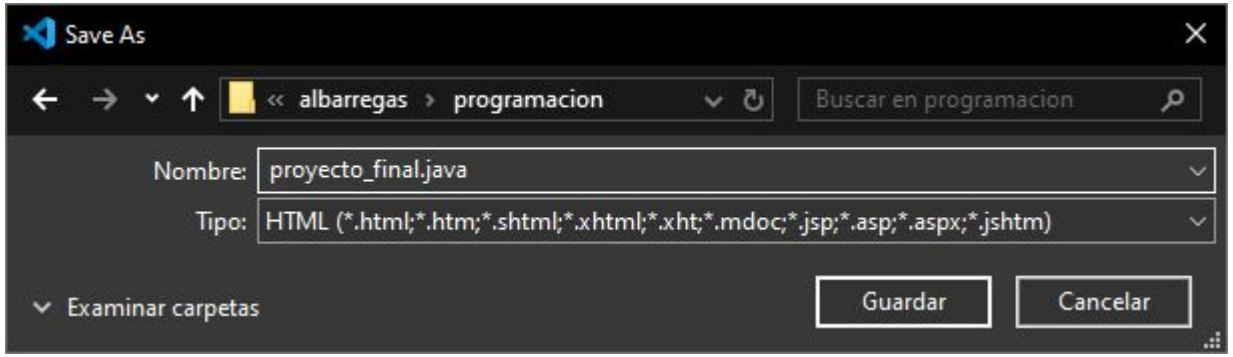


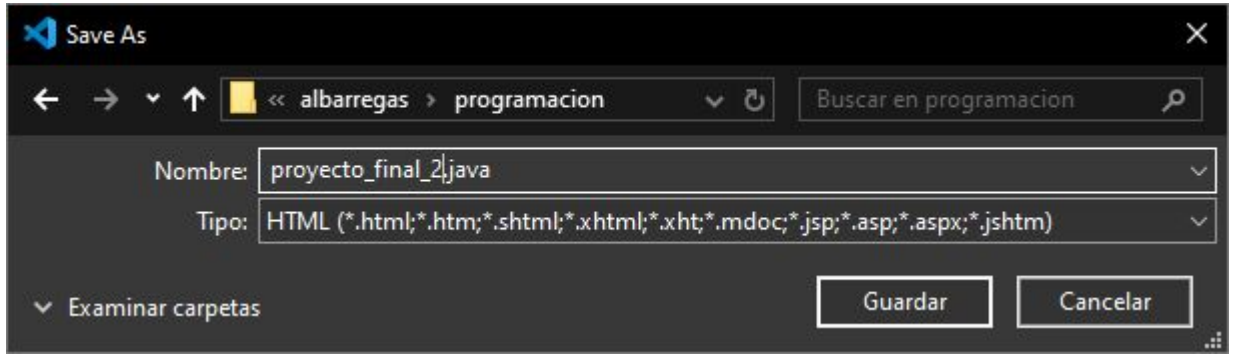
1º DAM/DAW

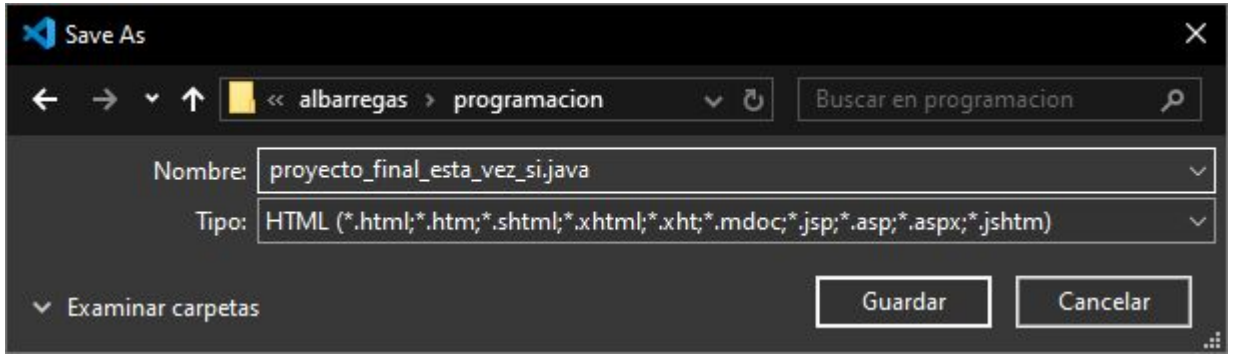
(2024-2025)

Guardando versiones de un proyecto









Sistemas de control de versiones (VSC)

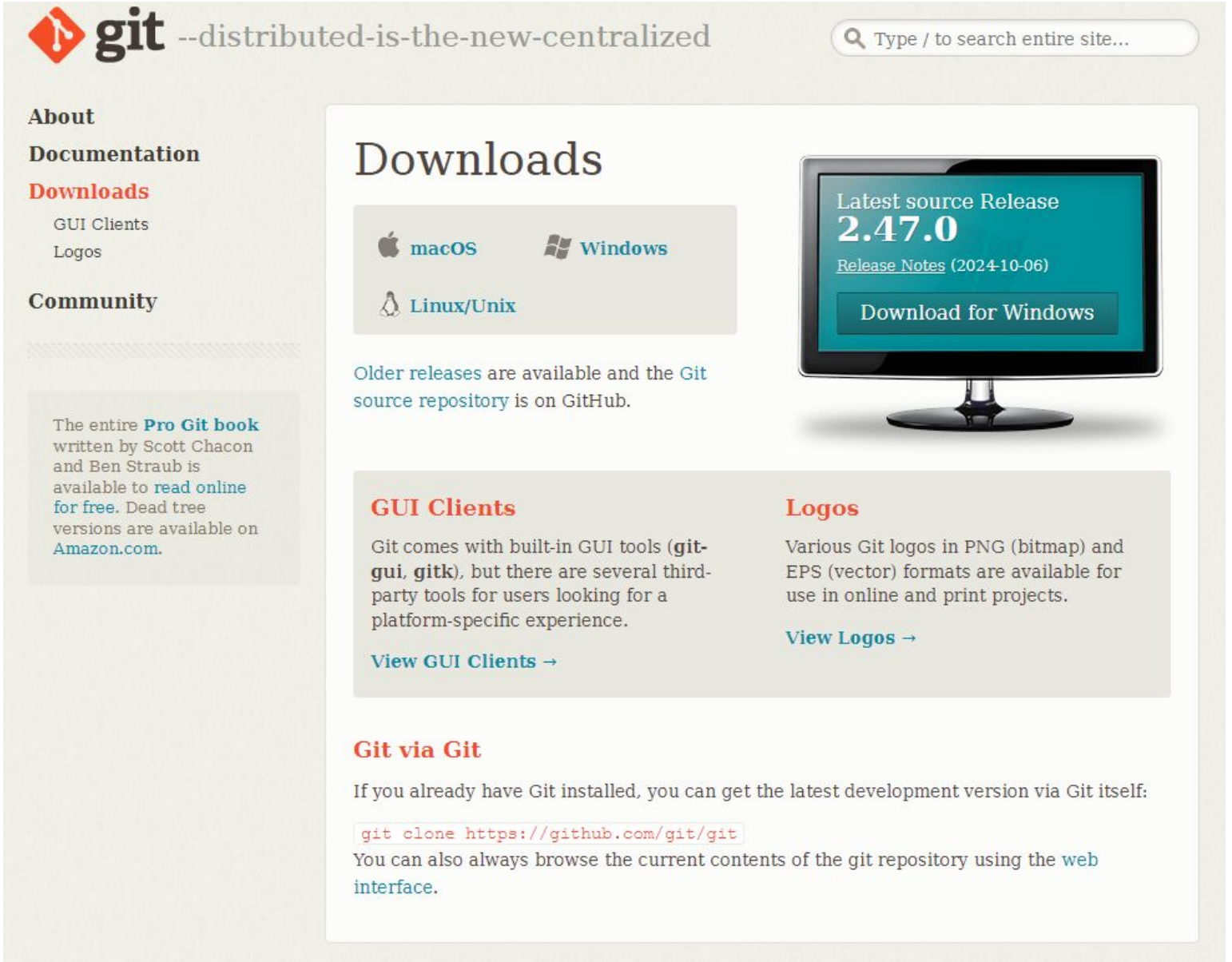
Mantiene el historial de los cambios realizados en un repositorio.

VSC vs Repositorio

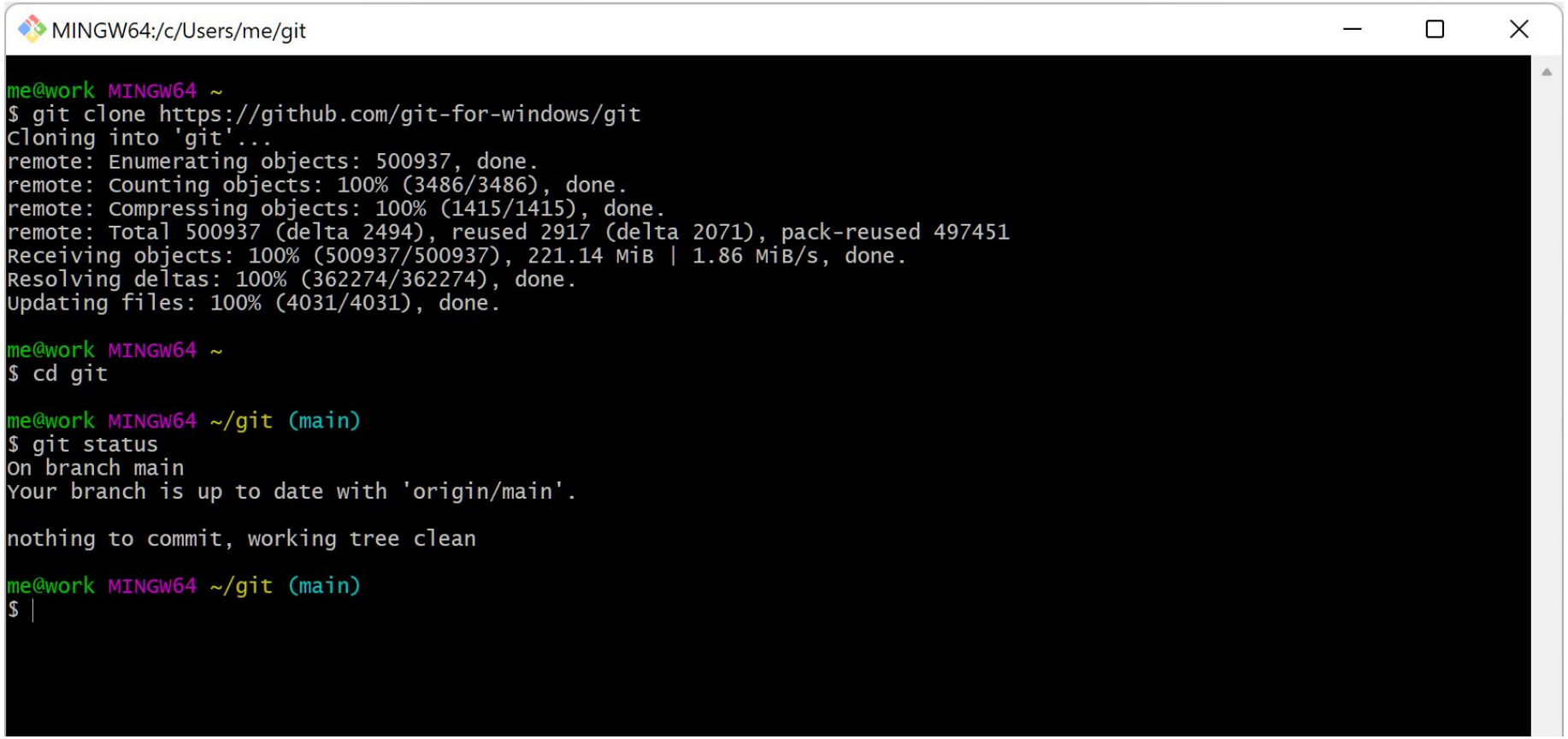
Comandos y algoritmos para el control de versiones.

Espacio de almacenamiento en la nube.
Repositorio remoto

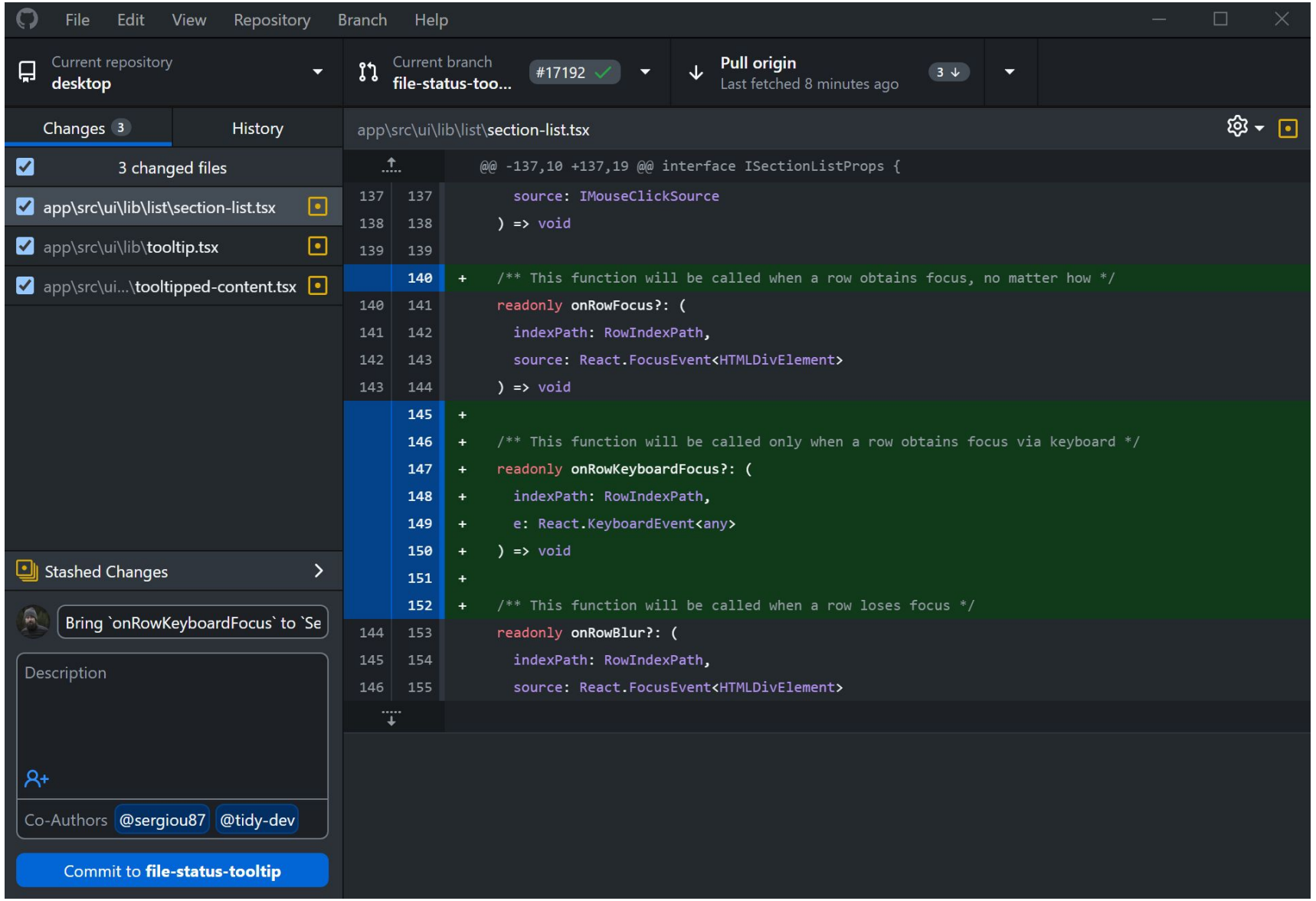
Descargar git



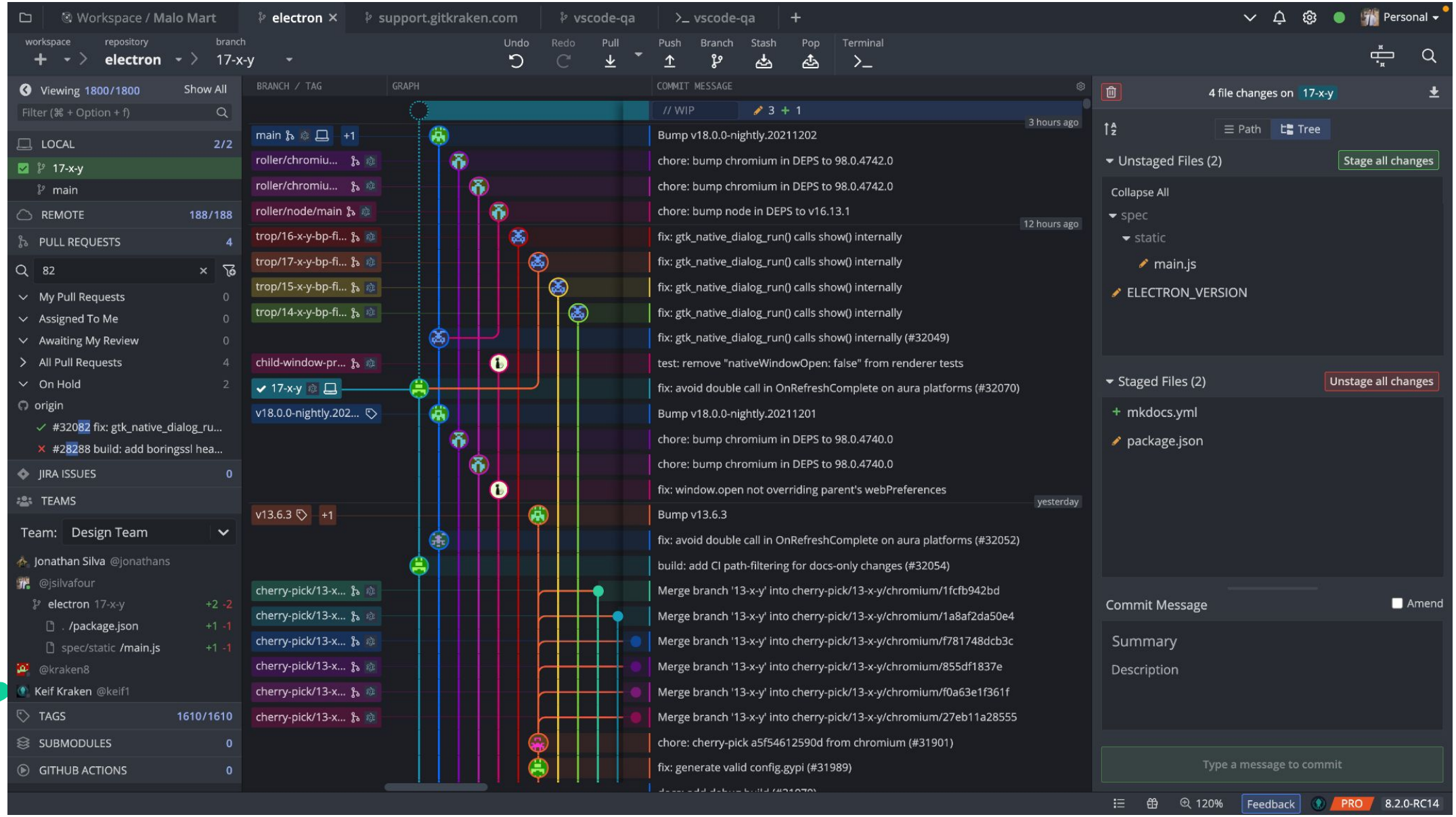
Herramientas - consola de git



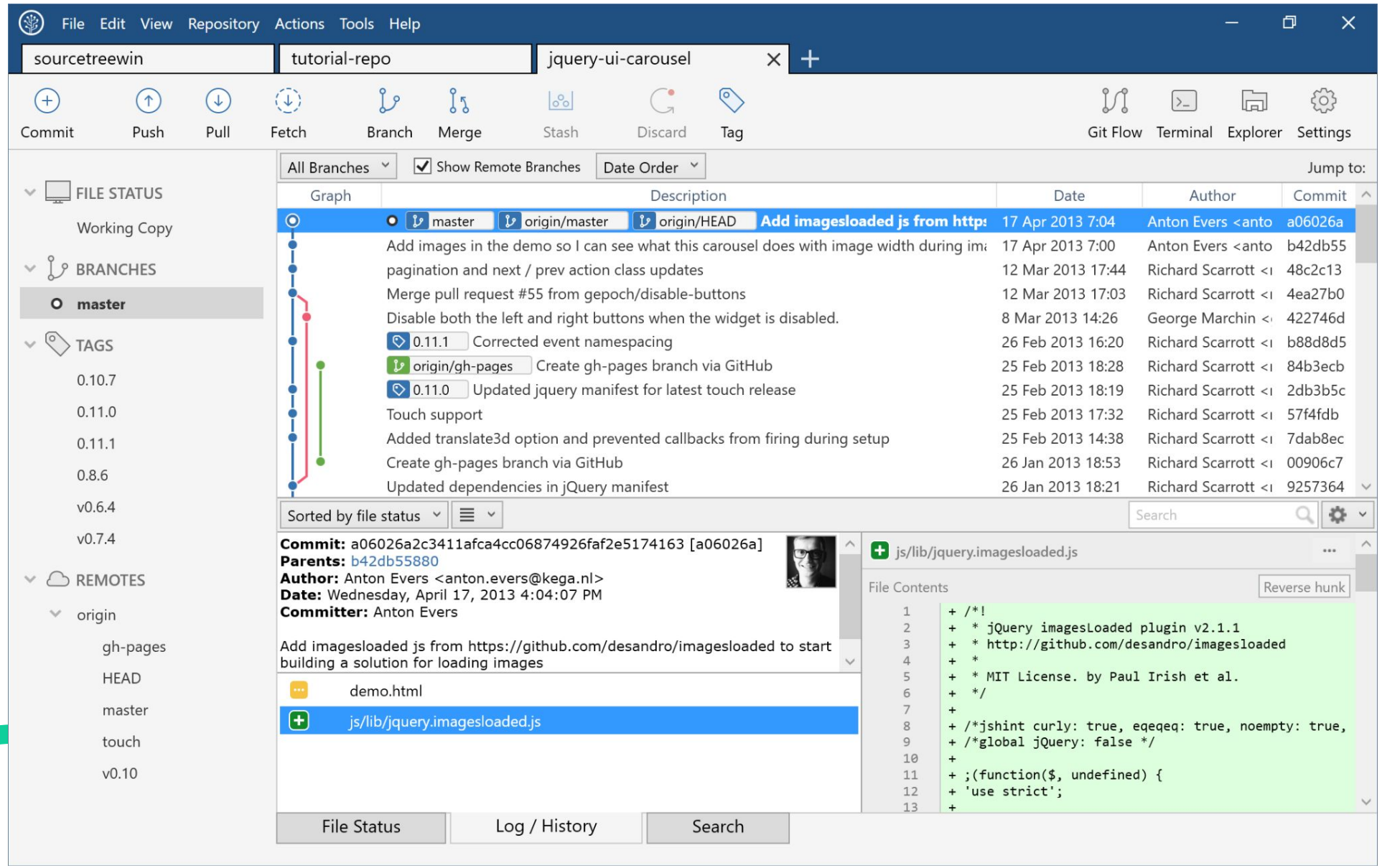
Herramientas - github desktop



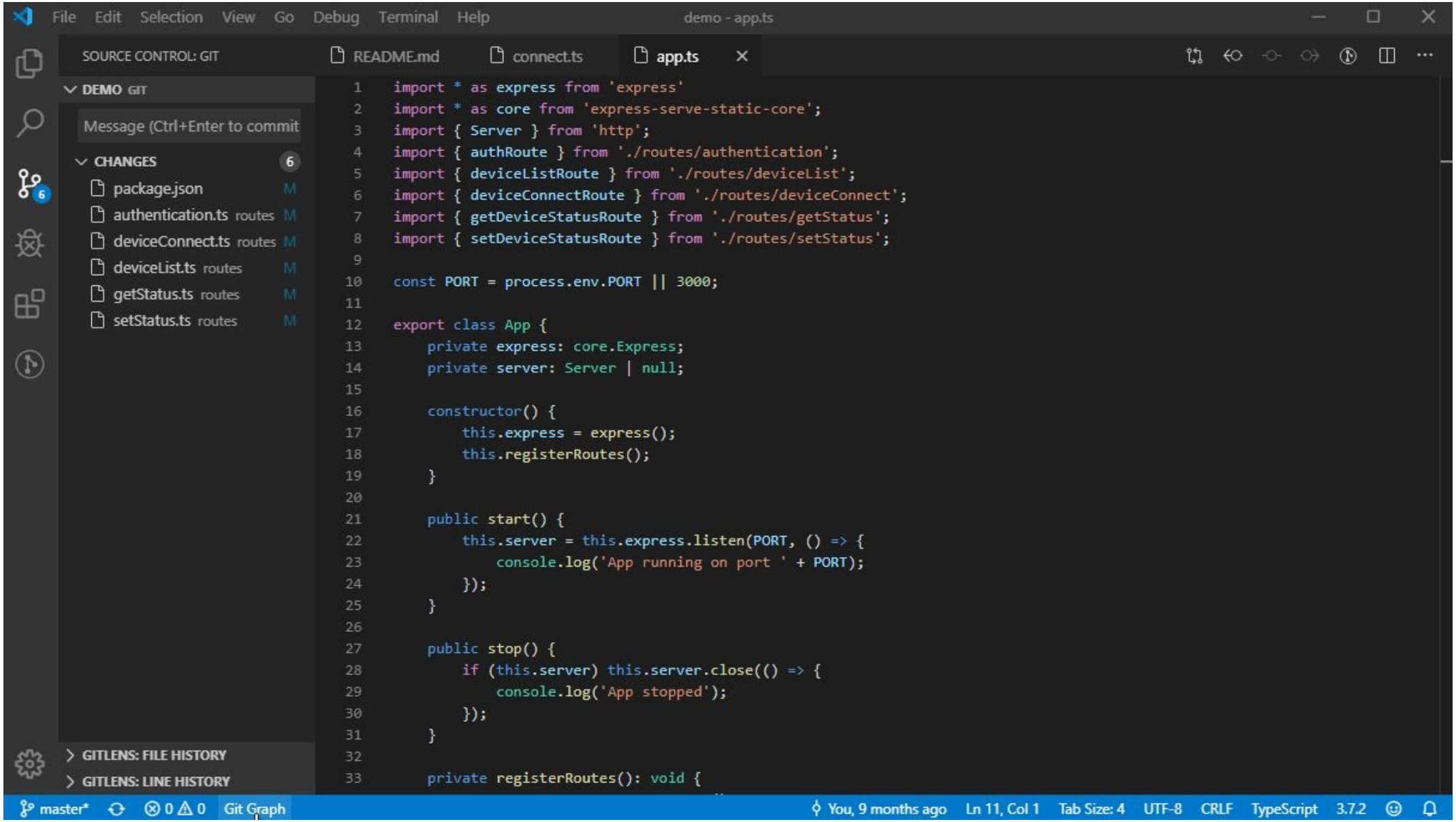
Herramientas - gitkraken (de pago)



Herramientas - sourcetree



Herramientas - gitgraph

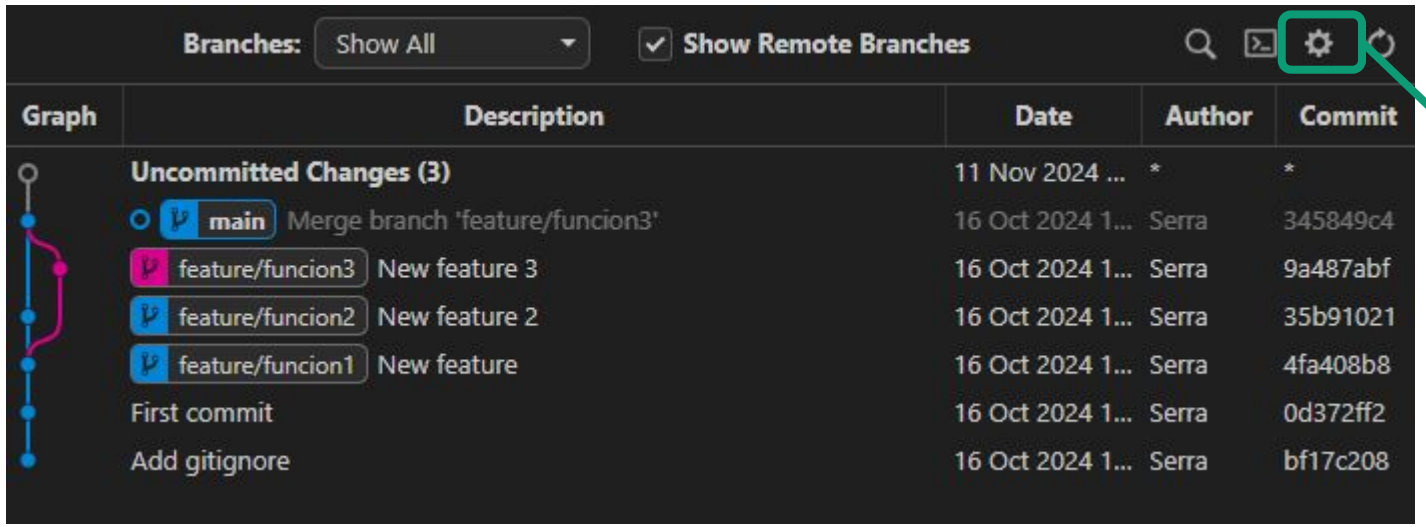


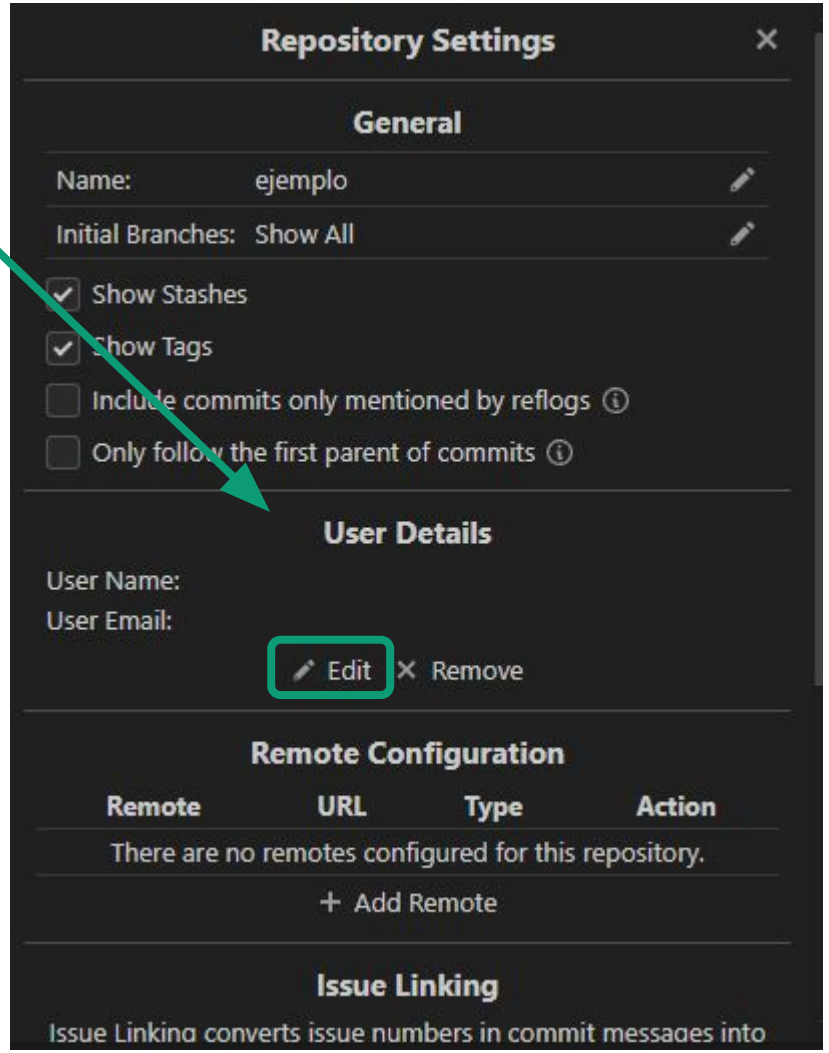
Configuración inicial de GIT

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

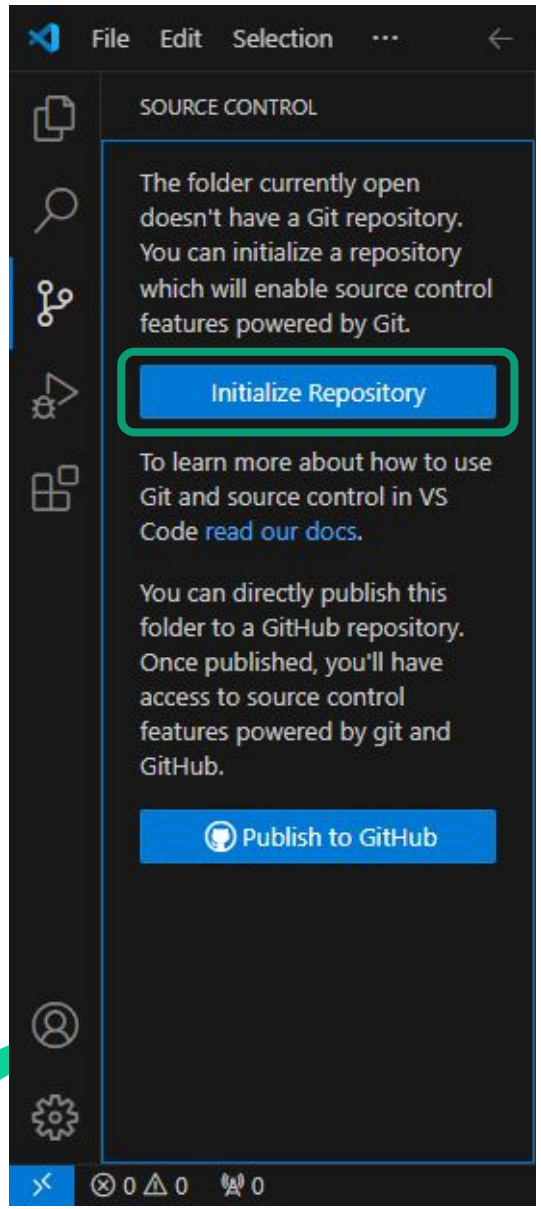
```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
```

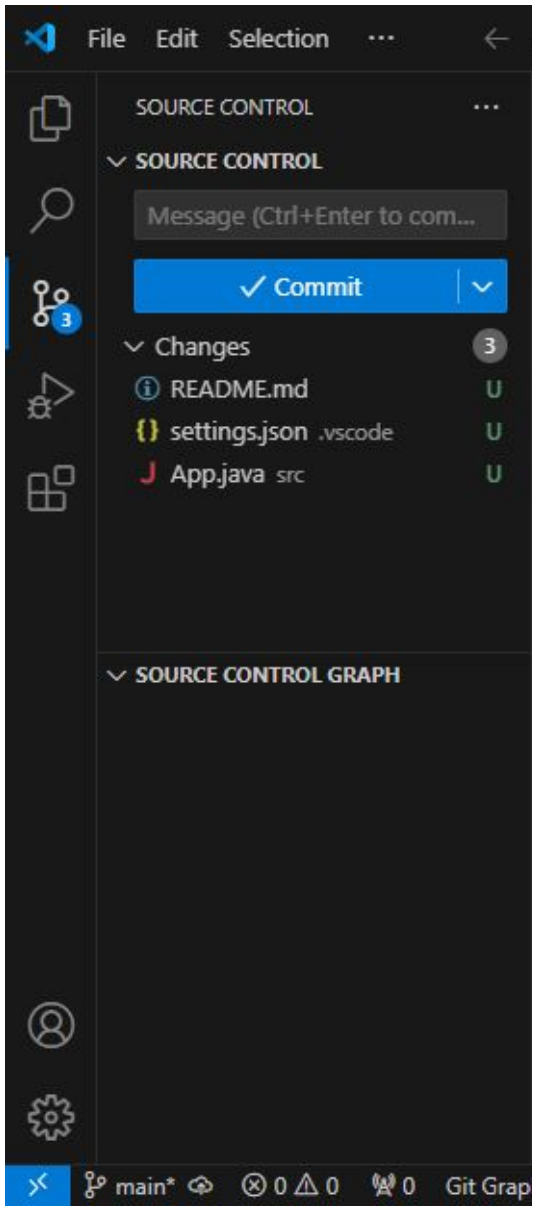
Configuración inicial de GIT - GitGraph

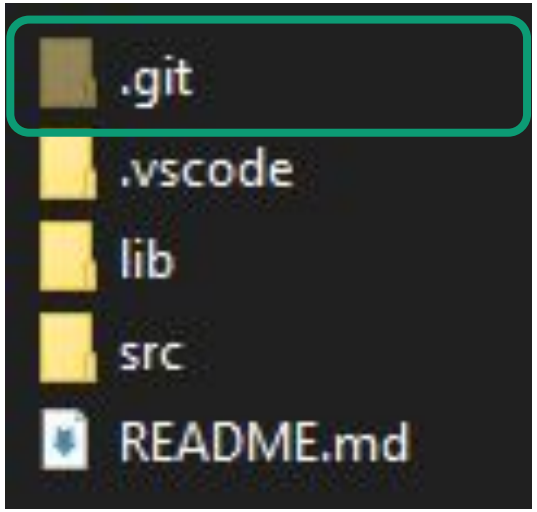




Crear un repositorio

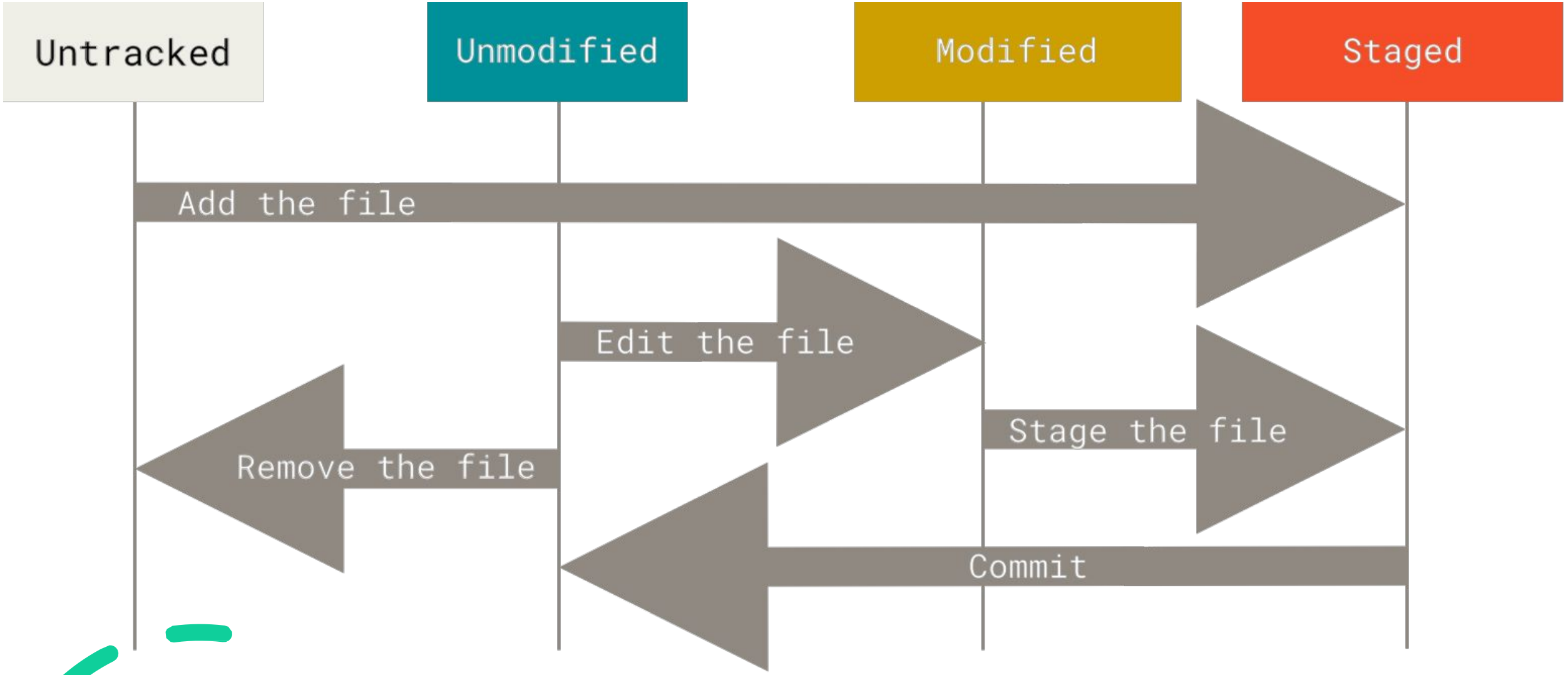






Aparece una carpeta oculta con la información del repositorio

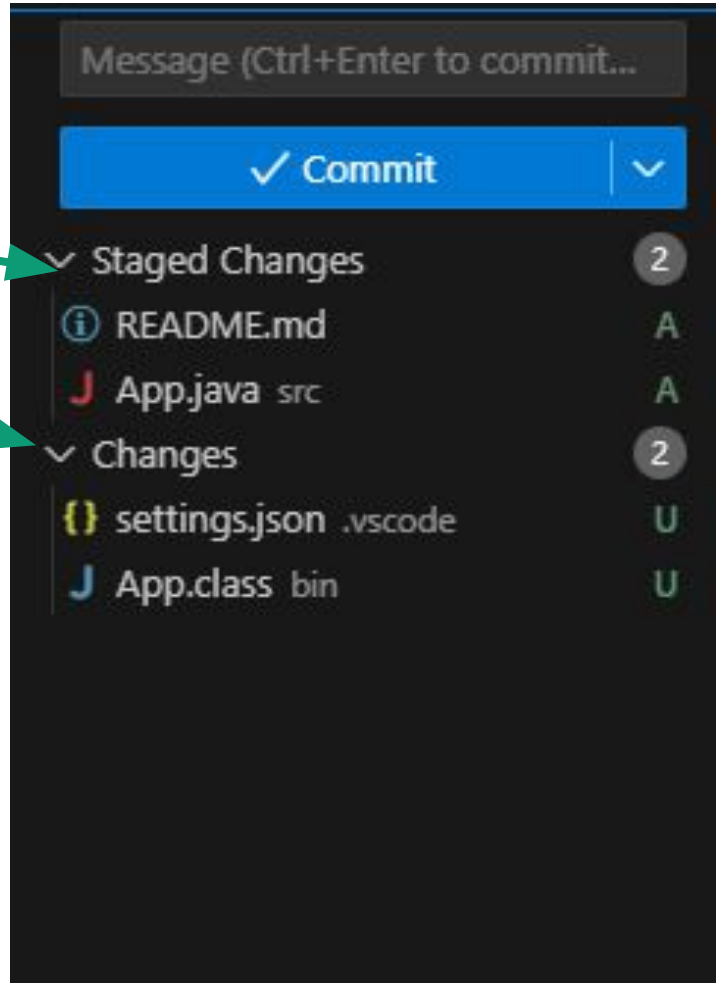
Estados de los archivos en un repositorio



Añadir archivos

Ficheros añadidos

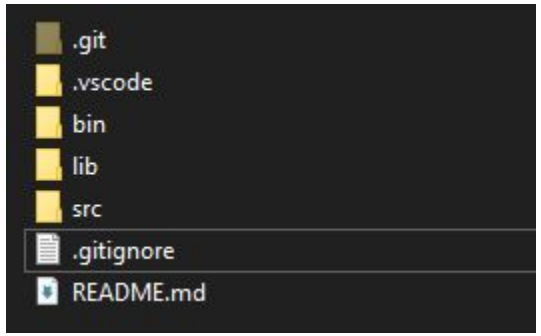
Ficheros sin añadir



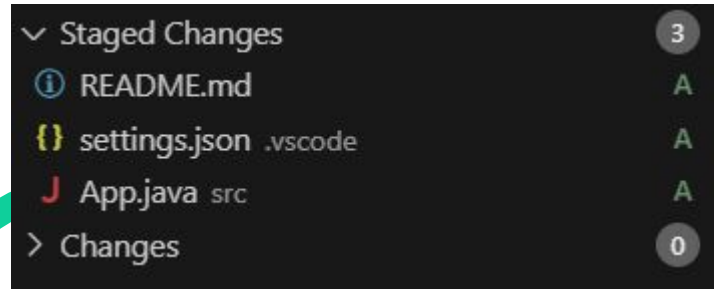

```
#Añadir un archivo al área de preparación
git add <nombre_del_archivo>
#Añdir todos los archivos modificados
git add .
```

.gitignore

Se añade en la raíz del repositorio



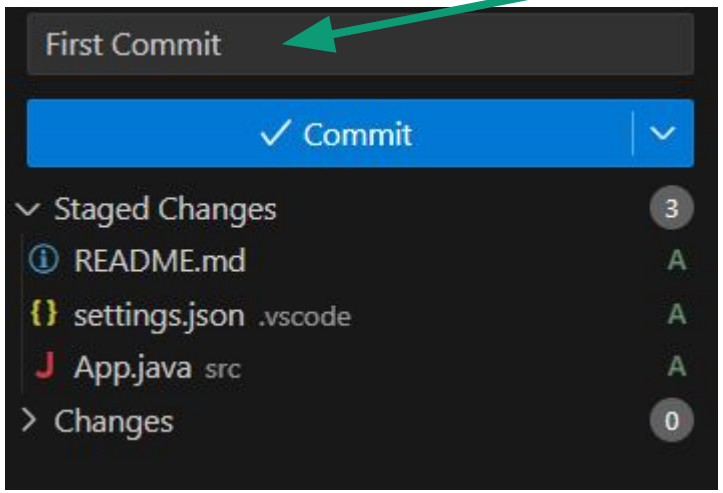
Los ficheros (muchas veces se pone un fichero genérico con una extensión) del gitignore no se almacenan en el repositorio.
Los .class desaparecen de la lista de cambios

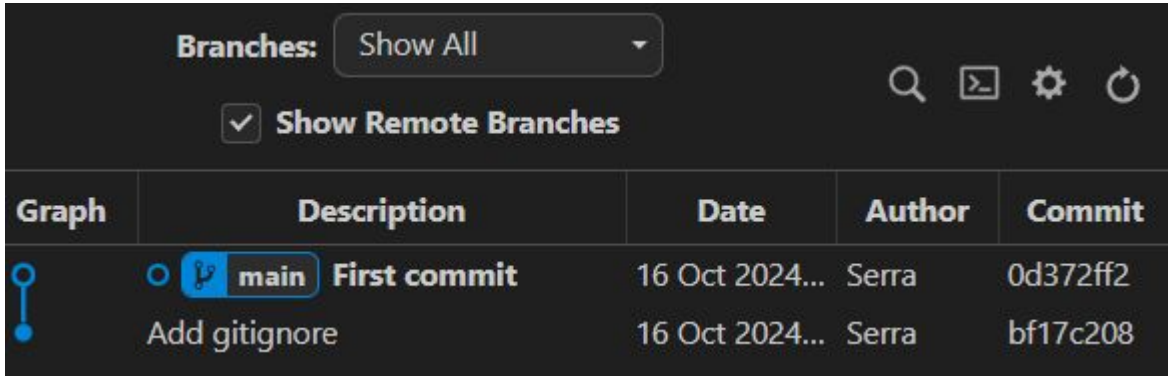



```
# Compiled class file
*.class

# Log file
*.log

# BlueJ files
*.ctxt

# Mobile Tools for Java (J2ME)
.mtj.tmp/

# Package Files #
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# virtual machine crash logs, see
http://www.java.com/en/download/help/error_hotspot.xml
hs_err_pid*
replay_pid*
```

Crear un commit

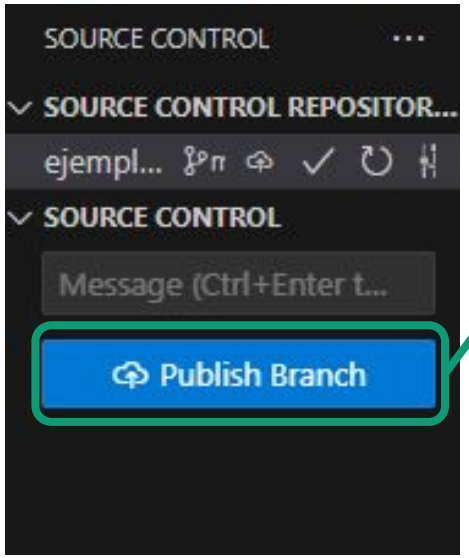
Mensaje de commit

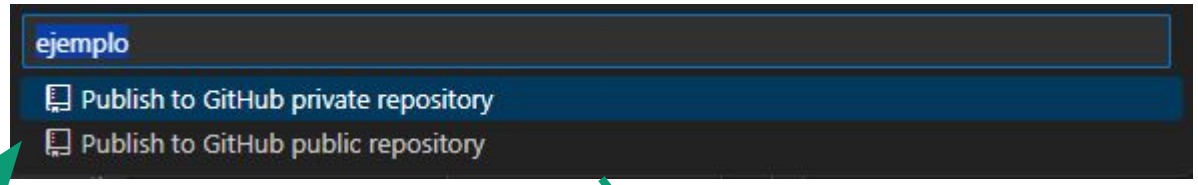


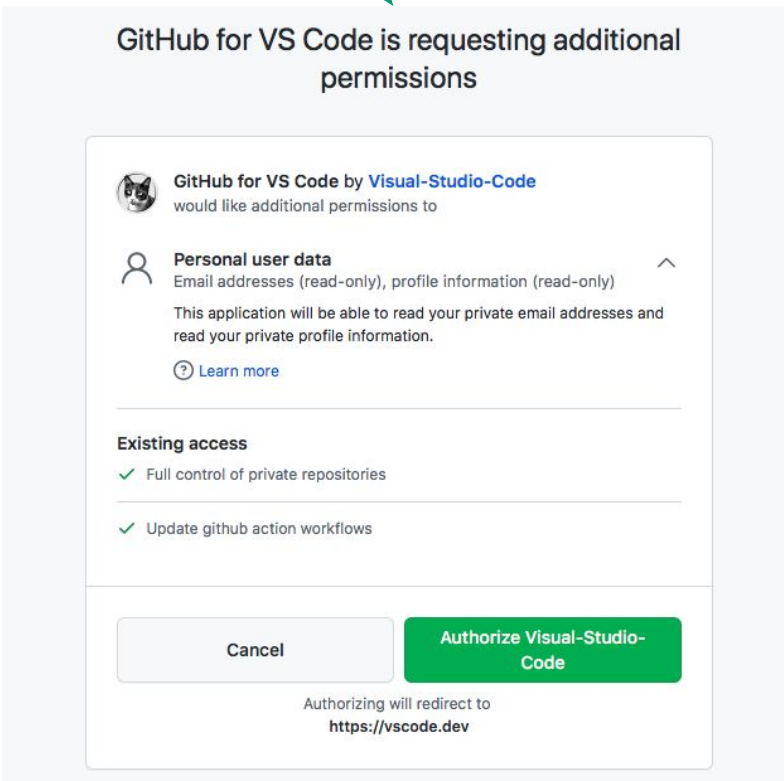


Un commit es un punto de guardado del repositorio, que se crea con los ficheros preparados en la staged area

Subir una copia al repositorio remoto



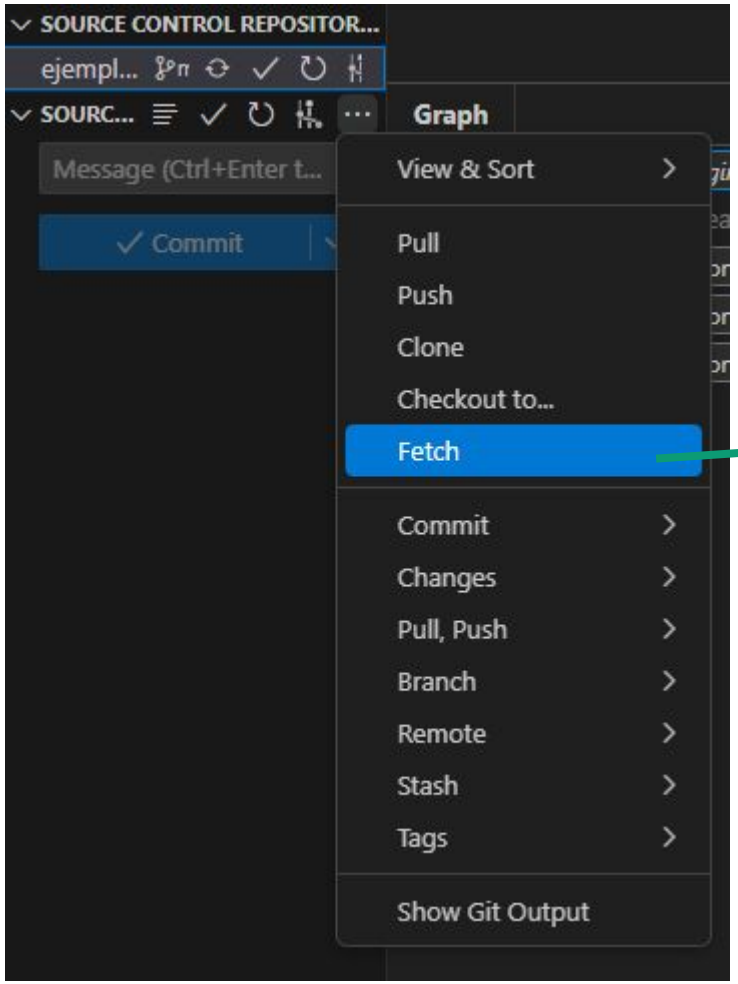


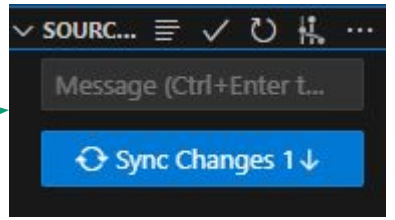


Comprobar y traer cambios del remoto

```
git fetch
```

```
git pull
```

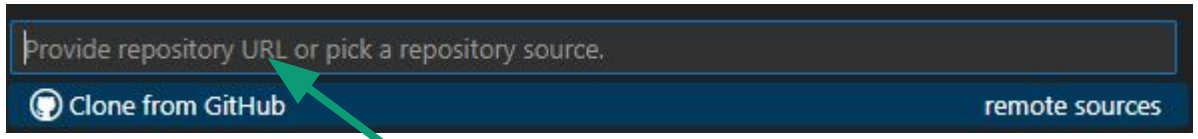




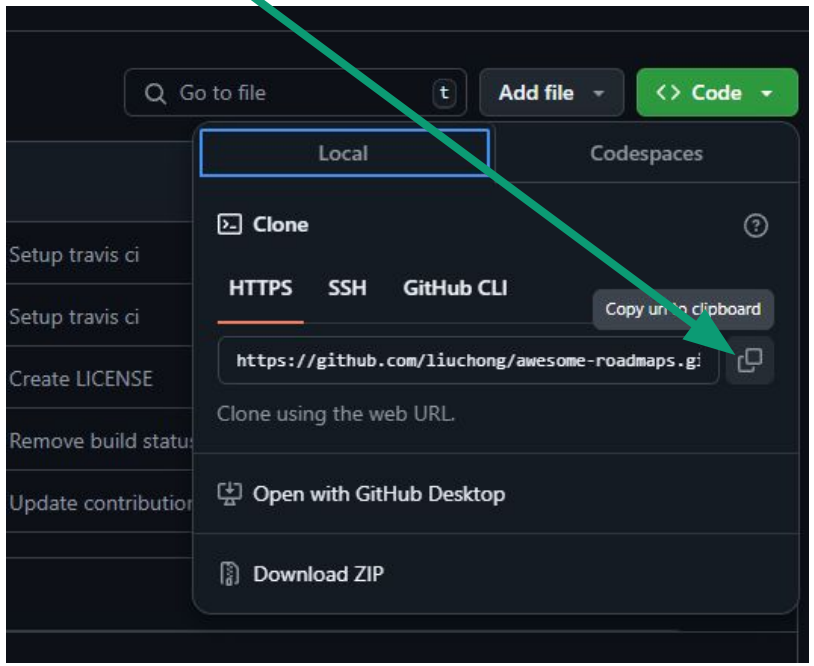
Fetch: Comprobar si hay cambios en el remoto
Pull: Traer cambios del remoto
Sync: Subir cambios pendientes + comprobar cambios + traer cambios

Clonar repositorio

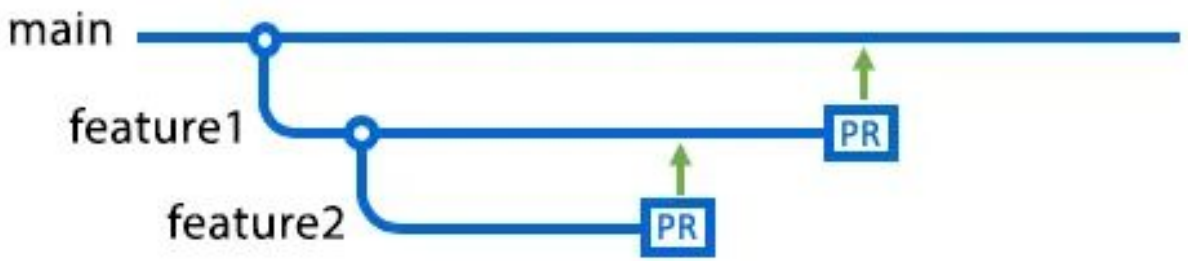
Show and run commands



Se puede clonar un repositorio público mediante el enlace. Si se es dueño o se participa en un repo, aparecen directamente en vscode en la interfaz de clonado.

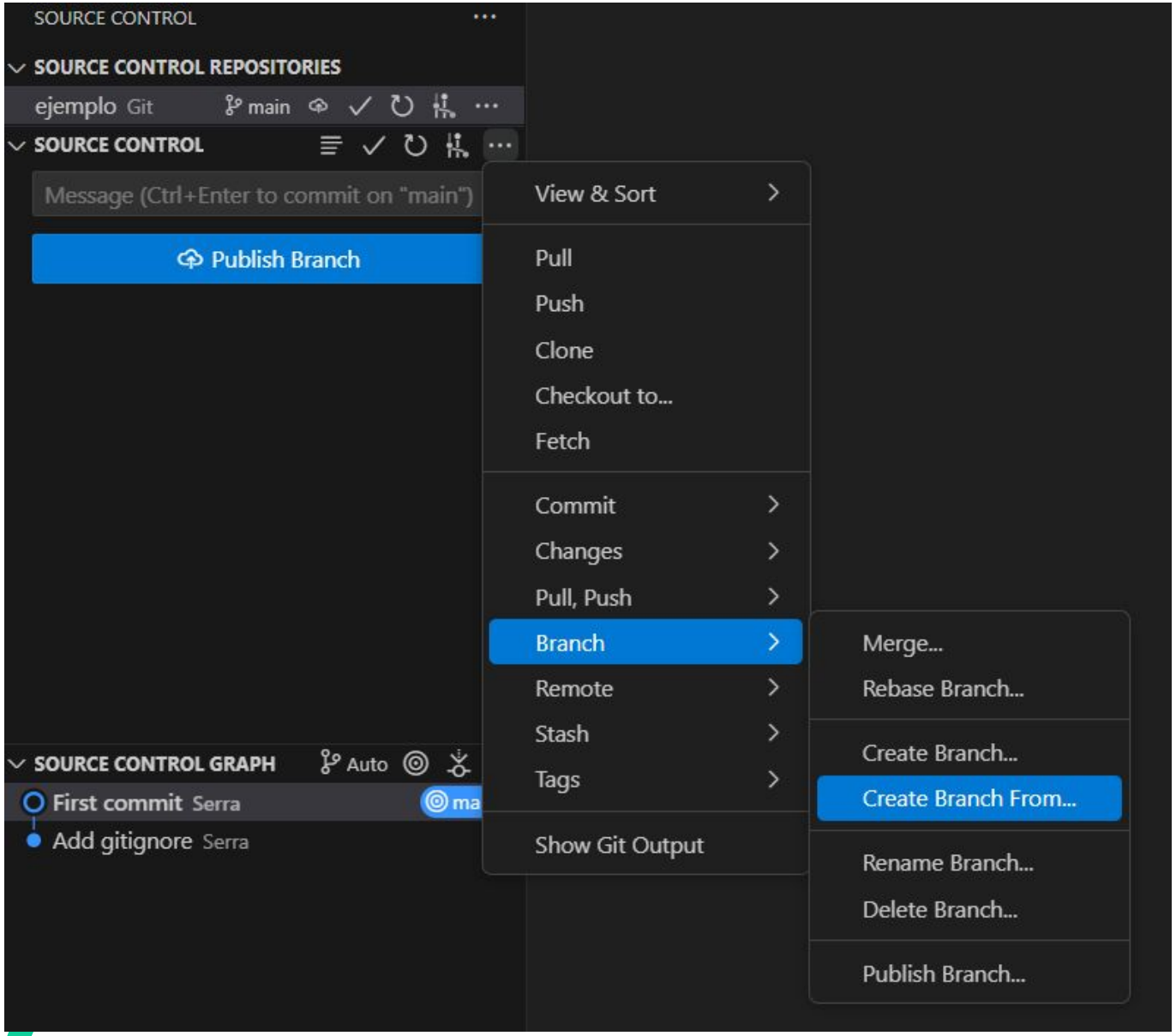


Ramas



Las ramas permiten desarrollar características, corregir errores, o experimentar sin afectar al resto del proyecto

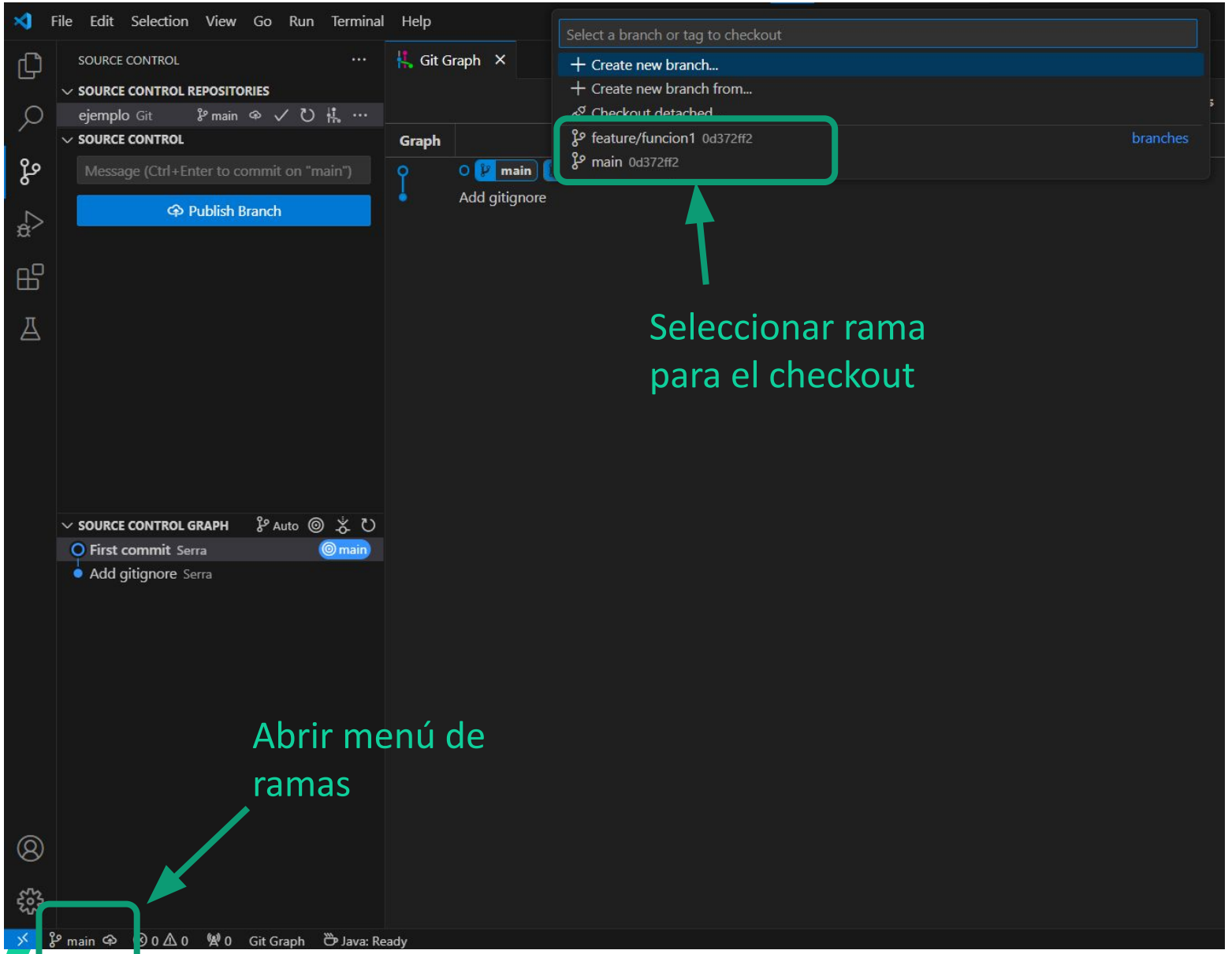
Crear una rama

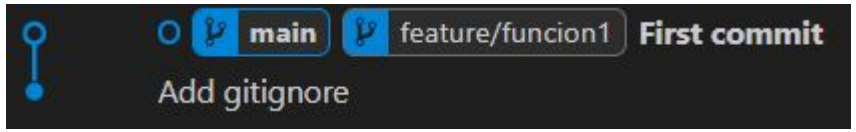



```
git branch feature/funcion1
```

Por defecto existe una rama llamada **main**, donde estará el programa listo para despliegue.
Se recomienda tener una rama llamada **develop**, donde se realiza el desarrollo.
Las distintas funcionalidades del proyecto se realizan en ramas de tipo **feature**.

Cambiar de rama

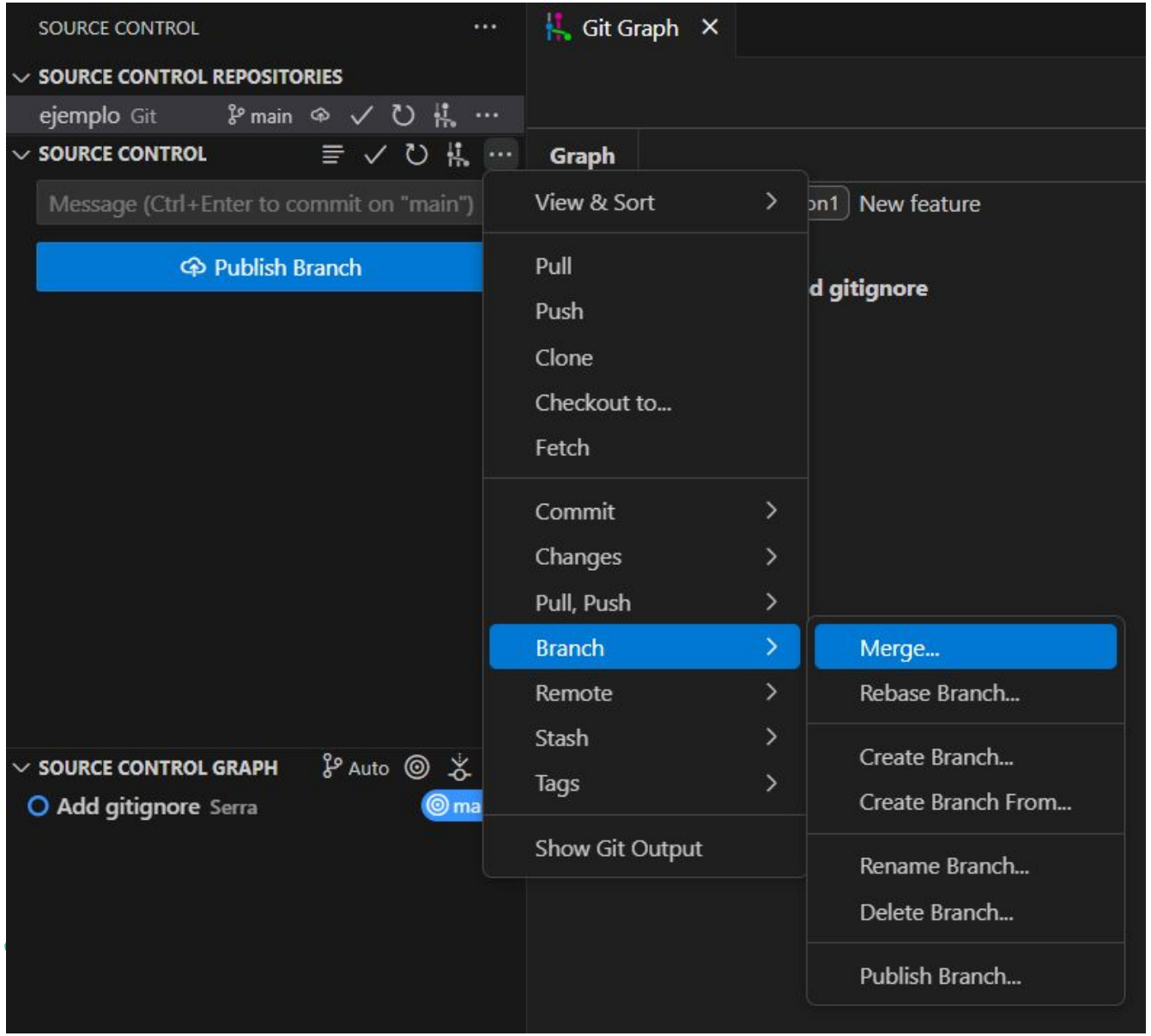


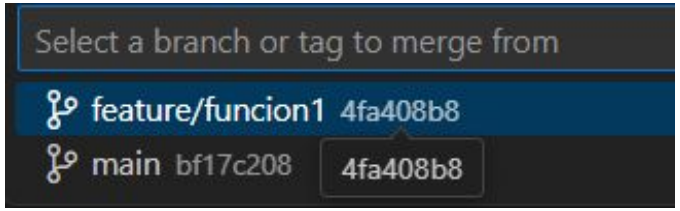


O hacer doble click en la interfaz gráfica de git graph

```
git checkout feature/funcion1
```

Mezclar ramas

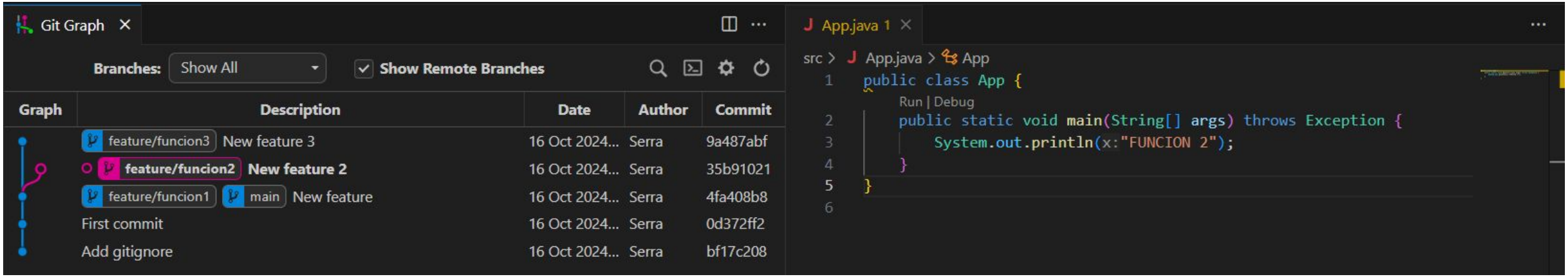


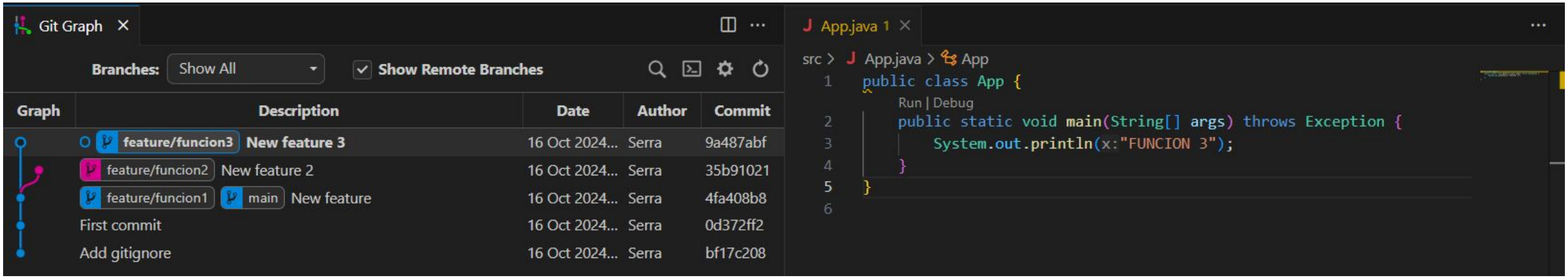


Para llevar los cambios de una rama a la rama main

- Checkout de main
- Seleccionar la opción merge
- Seleccionar la rama desde la que queremos traer los cambios

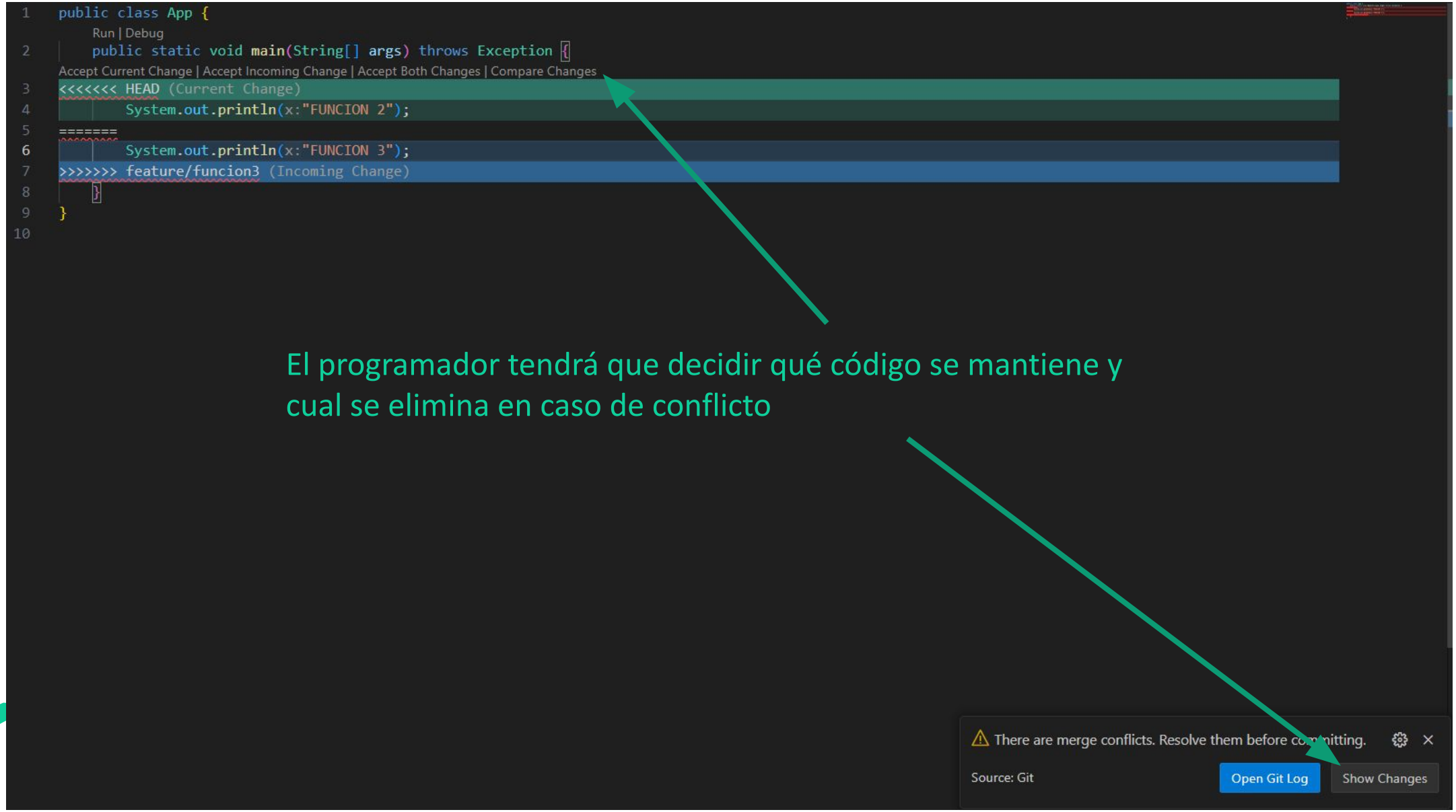
Conflictos





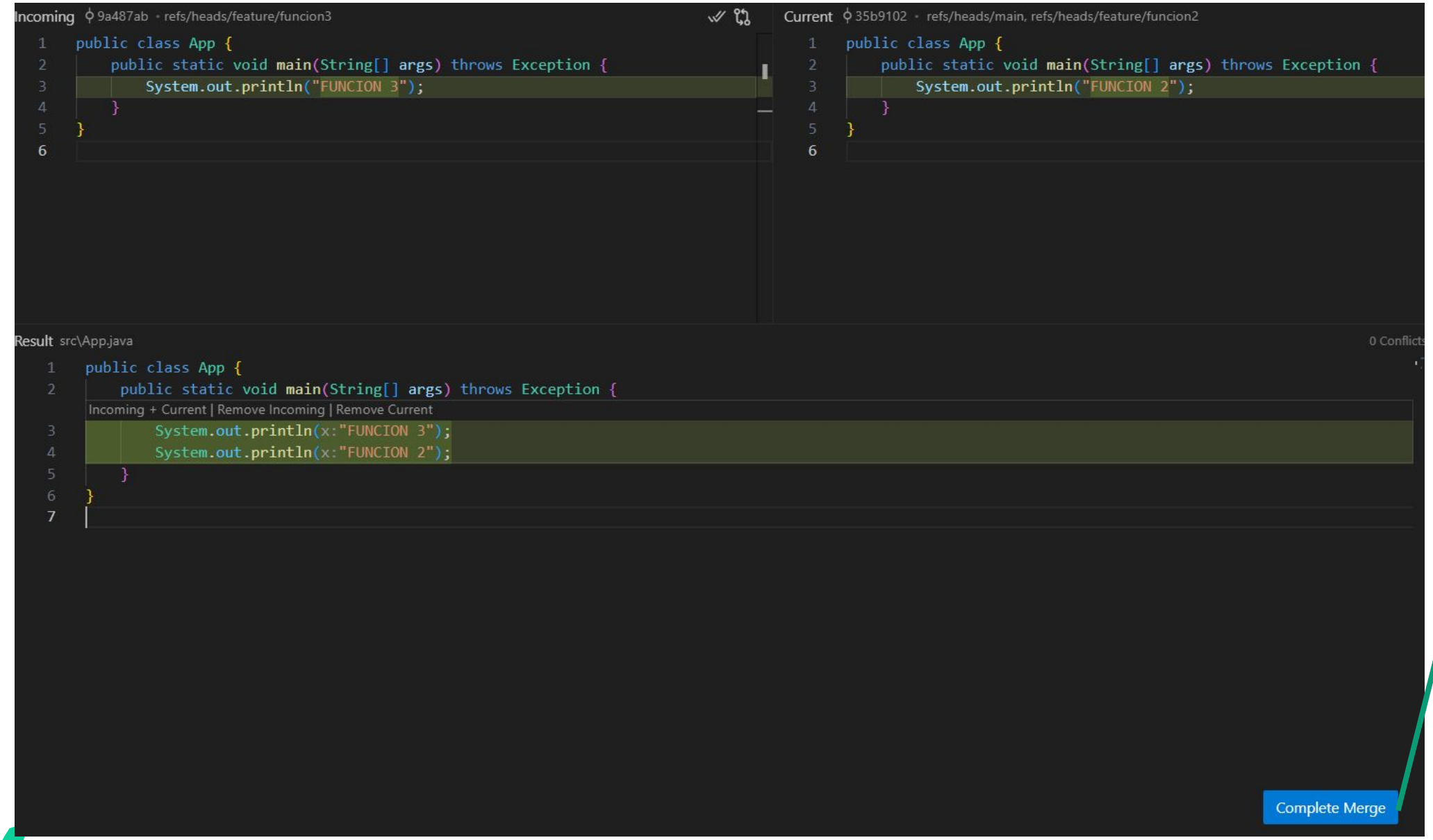
Cuando se trabaja simultáneamente en dos ramas distintas sobre los mismos archivos, surge un conflicto a la hora de mezclar ramas

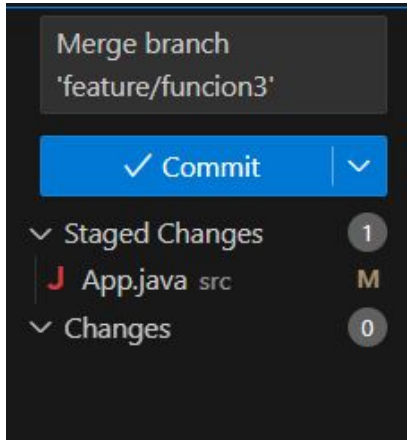
Conflictos



El programador tendrá que decidir qué código se mantiene y cual se elimina en caso de conflicto

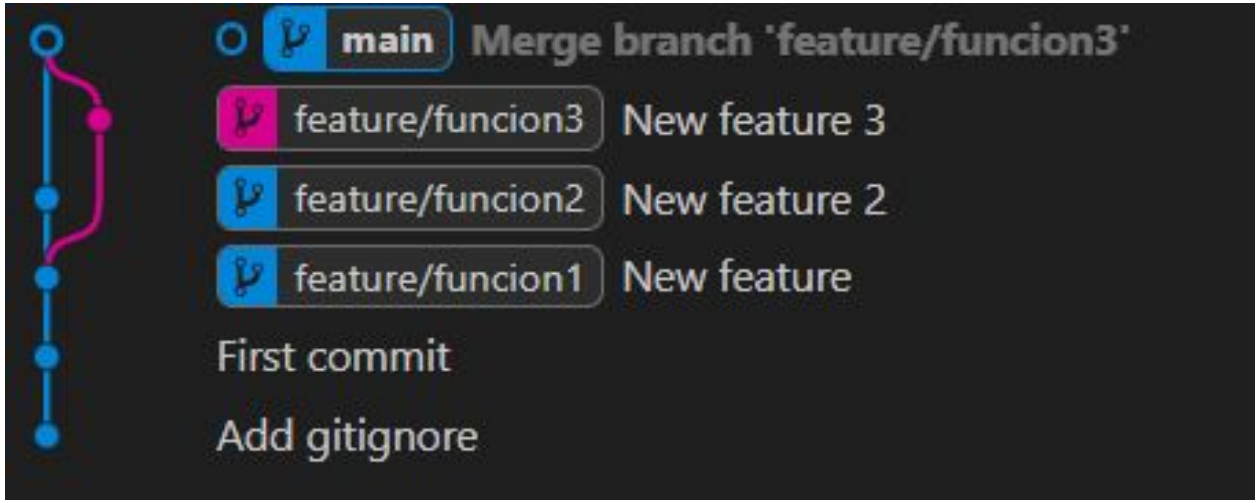
Conflictos





Tras revisar los cambios, habrá que hacer un commit

Conflictos

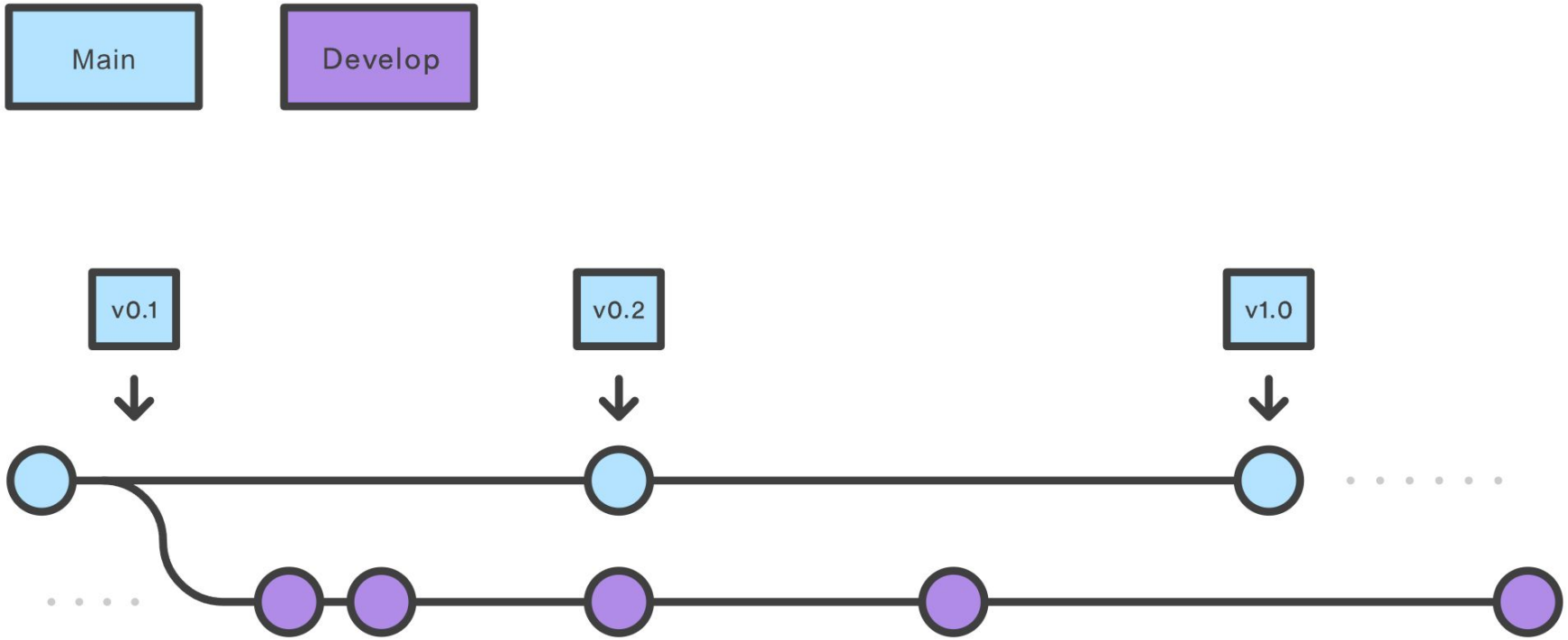


Al final del proceso de merge, tenemos en main tanto los cambios programados en funcion2 como en funcion3

Git Flow

Ramas principales y de desarrollo

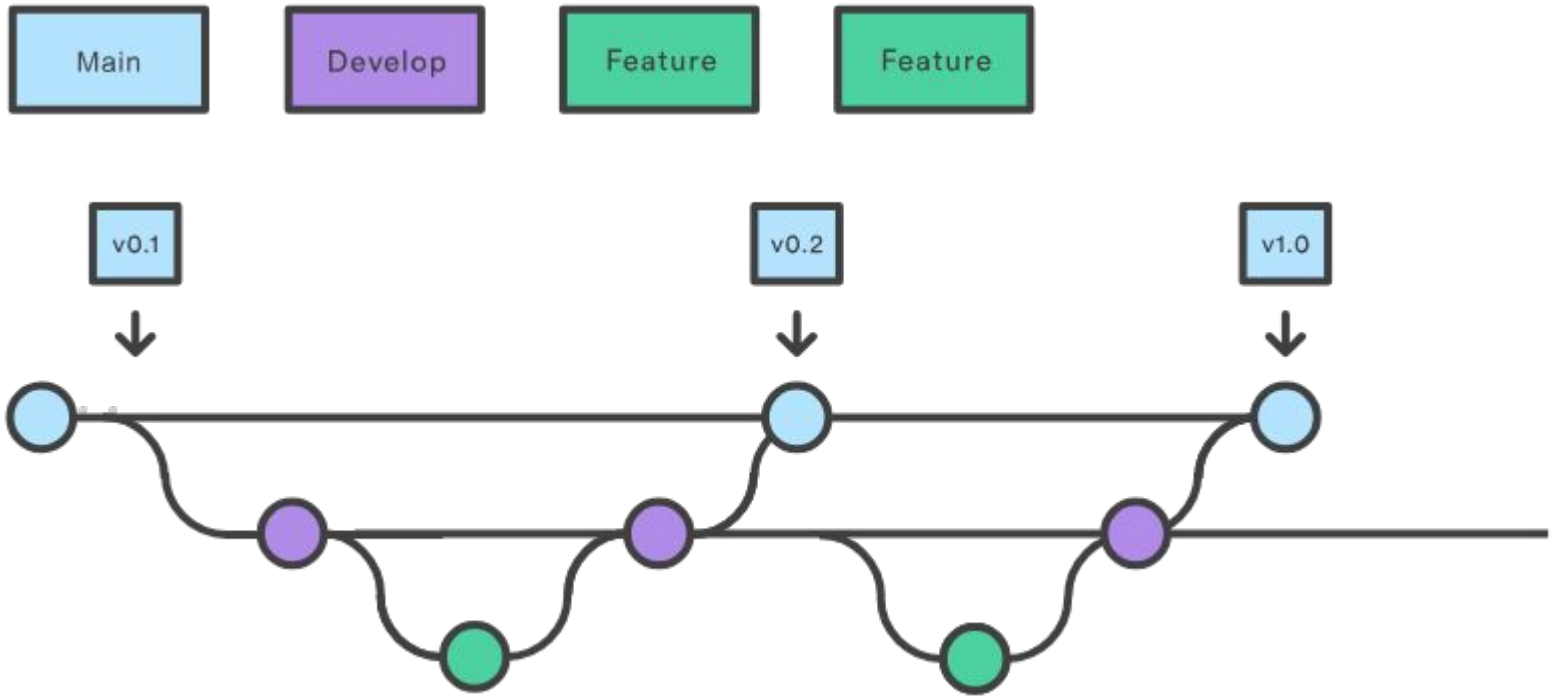
En lugar de una única rama main, este flujo de trabajo utiliza dos ramas para registrar el historial del proyecto. La rama main o principal almacena el historial de publicación oficial y la rama develop o de desarrollo sirve como rama de integración para las funciones. Asimismo, conviene etiquetar todas las confirmaciones de la rama main con un número de versión.



Git Flow

Ramas de función

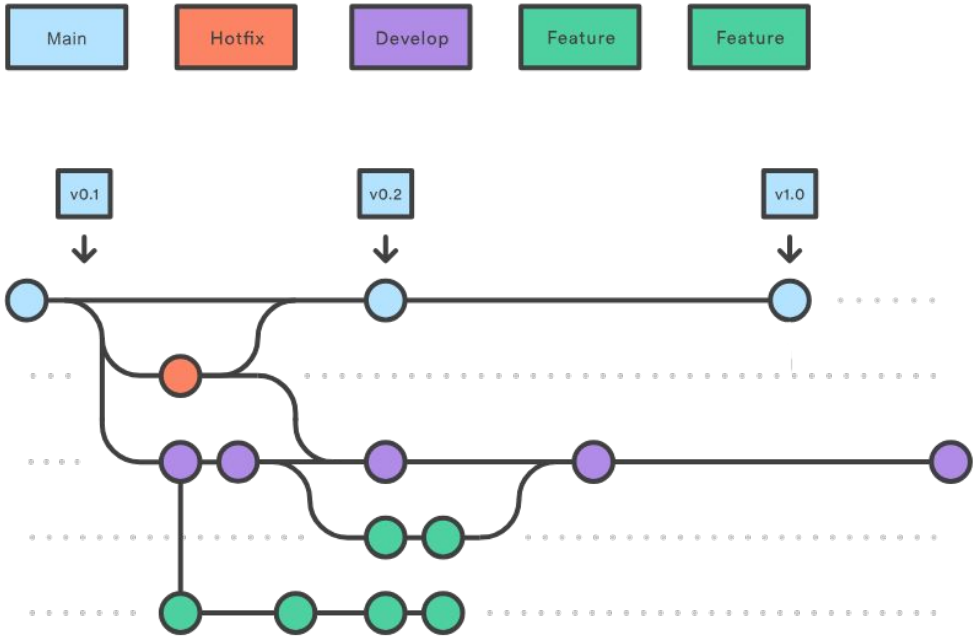
Todas las funciones nuevas deben residir en su propia rama, que se puede enviar al repositorio central para copia de seguridad/colaboración. Sin embargo, en lugar de ramificarse de main, las ramas feature utilizan la rama develop como rama primaria. Cuando una función está terminada, se vuelve a fusionar en develop. Las funciones no deben interactuar nunca directamente con main.



Git Flow

Ramas de corrección

Las ramas de mantenimiento, de corrección o de hotfix sirven para reparar rápidamente las publicaciones de producción. Las ramas hotfix son muy similares a las ramas release y feature, salvo por el hecho de que se basan en la rama main y no en la develop. Esta es la única rama que debería bifurcarse directamente a partir de main. Cuando se haya terminado de aplicar la corrección, debería fusionarse en main y develop, y main debería etiquetarse con un número de versión actualizado.



Git Flow

El flujo general de Git Flow es el siguiente:

1. Se crea una rama develop a partir de main.
2. Se crean ramas feature a partir de la rama develop.
3. Cuando se termina una rama feature, se fusiona en la rama develop.
4. Cuando se llega a una versión estable en develop, se integra en main.
5. Si se detecta un problema en main, se crea una rama hotfix a partir de main.
6. Una vez terminada la rama hotfix, esta se fusiona tanto en develop como en main.